

HDFS 스토리지 Auto-Tiering YARN 서비스

분산시스템및컴퓨팅(2026-1)

202111268 김태수

202111351 이준혁

202211404 황서진

202315324 최세진

1 제안 배경 및 필요성

현재 HDFS는 파일 블록을 이기종 스토리지 간에 이동시킬 수 있는 메커니즘 자체는 제공하고 있다. 하지만 Amazon S3와 같은 클라우드의 스토리지 서비스들이 기본적으로 제공하는 데이터 Lifecycle 자동 전환 기능과 달리, HDFS에는 자동으로 분석하고 감지하여 저비용 스토리지로 이전해 주는 기능이 없다.

이에 Apache Ozone이라는 대안이 존재하지만, 이미 HDFS에 대규모로 축적된 데이터를 이전하는 것은 기업이나 관리자 입장에서 비용과 리스크 측면에서 매우 큰 부담이다. 이로 인해 HDFS를 사용한다면 관리자가 별도의 주기적인 이주 스크립트를 운영해야 하며, 그렇지 않을 경우 콜드 데이터가 고비용의 자원을 불필요하게 점유하여 점차 인프라의 총 소유 비용을 증가시키는 문제가 발생하게 된다.

따라서 본 프로젝트는 최근 접근 시점을 바탕으로 분석하여 알맞은 스토리지 정책으로 자동 전환해주는 Auto-Tiering YARN 서비스를 구현한다. 이를 통해 사용자 관점에서는 운영 비용 최적화를 이룰 수 있고, 서비스 관점에서는 기존 Hadoop 클러스터의 YARN 프레임워크를 활용하여 외부 인프라 없이 자원 격리 및 고가용성을 보장하며 YARN에 오토 티어링 서비스만을 추가해 인프라 효율을 극대화할 수 있다.

2 과제 목표

2.1 FSImage 오프라인 파싱을 통한 메타데이터 분석 및 스코어링

Active NameNode의 글로벌 락 및 RPC 오버헤드를 방지하기 위해, 스냅샷 처리된 FSImage를 비동기적으로 스캔한다. 네임노드는 단일 글로벌 락을 사용하므로, 외부의 무차별적인 API 호출은 클러스터 마비를 초래할 수 있다. 이를 방지하기 위해 FSImage 파일을 OIV(Offline Image Viewer)로 CSV 변환 후, Python/Java를 통해 용량과 방치 시간을 결합한 우선순위 점수를 산출한다. 우선순위가 높은 콜드 데이터를 우선적으로 식별하여 리스트업한다.

2.2 우선순위 기반 배치(Batch) 데이터 이동

리스트업된 파일에 대해서 HdfsAdmin API로 스토리지 정책을 변경한다. 이후 satisfyStoragePolicy API를 호출해 물리적 이동을 트리거한다. 이때 한번에 모든 파일을 이동시키지 않고, 우선순위 점수에 따라 일정 개수(batch)씩 나누어 이동시킨다. 또한 클러스터 워크로드가 높은 주간 시간대에는 배치 크기를 보수적으로 제한하여 사용자 쿼리 성능을 보호하고, 가용 리소스가 풍부한 야간 시간대에는 배치 크기를 확대하여 티어링 효율을 극대화한다. 각 배치 사이에는 지정된 대기 시간을 두어 디스크 I/O 및 DataNode 간 네트워크 대역폭 포화를 방지하고 클러스터 안정성을 확보한다.

3 주요 내용 및 개발 범위

3.1 메타데이터 분석 및 우선순위 파이프라이닝

- FSImage 파싱: OIV를 이용해 필요한 정보를 추출 및 파싱
- 스코어링 및 적재: 변환된 파일에서 경로, 크기, 최종 접근 시간 정보를 추출하고 우선순위 점수를 산출하여 적재

3.2 서비스 안정성 및 부하 제어

- 배치 기반 워크로드 제어: 대규모 파일 이동 시 발생하는 NameNode RPC 오버헤드와 네트워크 대역폭 점유를 방지하기 위해 가변 배치 스케줄링 적용
- 논리적 대역폭 관리: 배치 간 대기 시간 및 동시 실행 수 제한 로직을 통해 인프라 부하 조절
- External SPS 활용: 네임노드 내부에서 동작하는 Internal SPS 대신, 별도 데몬으로 구동되는 External SPS 모드를 사용하여 데이터 이동 처리가 클러스터 핵심 서비스 성능에 미치는 영향을 최소화

3.3 우선순위 티어링 알고리즘

- 용량 Filtering: 소형 파일은 이동 오버헤드 대비 용량 확보 이득이 적으므로 티어링 대상에서 제외하거나 우선순위를 매우 낮춰 시스템 리소스 낭비를 방지
- 스코어링 모델 설계: 스코어링 알고리즘을 설계해 우선순위 기반 티어링을 적용. 이때 가중치는 외부 YAML 설정 파일로 관리하여 운영 중에도 서비스 중단 없이 수정이 가능하도록 함.
- 시간대별 가변 정책: 클러스터 사용량이 많은 주간에는 보수적인 배치 사이즈를 유지하고, 사용량이

적은 야간에는 배치 크기를 확대하여 티어링 속도를 높이는 가변 운영 모드를 도입

3.4 실행 및 통합 모니터링 구현

- 비동기 이동 상태 추적: satisfyStoragePolicy API 호출 후 실제 물리적 블록 이동이 완료되었는지 주기적 샘플링 또는 SPS의 JMX Metrics를 수집하여 클러스터 전체의 처리량과 대기열을 모니터링하는 피드백 루프 구축
- 메타데이터 수집 안정화: Active NameNode의 부하를 피하기 위해 Standby NameNode로부터 FSImage를 수집
- 운영 모니터링: 티어별 용량 변화 현황, 우선순위별 이동 진행률, 대역폭 제한 통계 로그 분석 및 검증

4 기대효과

4.1 데이터 스토리지 운영 자동화 및 효율화

- 수동 개입 최소화: HDFS의 물리적 정책(All_SSD, One_SSD, Cold)과 매핑된 HOT, WARM, COLD의 3단계 스토리지 전환을 자동으로 수행
- 인프라 관리 효율 향상: 데이터의 최근 접근 시점 분석과 용량을 바탕으로 콜드 데이터를 저비용 스토리지로 자동 격리

4.2 정량적 데이터 관리 지표 확보

- 정책 전환 정확도: 설정된 임계 시간을 초과한 파일이 의도한 스토리지 정책으로 정확히 전환되는지 측정하여 시스템의 신뢰성을 검증
- 블록 이동 처리 성능: 실제 데이터 블록이 이기종 매체 간에 이관되는 시간을 측정하여 시스템의 성능 오버헤드를 분석

4.3 인프라 총 소유 비용(TCO) 절감

- 저장 비용 최적화: 고비용의 SSD 자원을 점유하던 콜드 데이터를 저렴한 스토리지로 이전함으로써 데이터 저장 비용 절감 효과를 극대화
- 추가 도입 비용 방지: 상용 클라우드(S3 등)나 별도의 오브젝트 스토리지로 데이터를 마이그레이션할 필요 없이, 기존 HDFS 환경 내에서 비용 효율적인 운영이 가능

4.4 시스템 안정성 및 고가용성 보장

- 리소스 보호: 배치 스케줄링과 모니터링을 통해 마이그레이션 중에도 기존 분산 서비스의 성능 저하를 방지

- 서비스 연속성: YARN Service Framework를 통해 특정 노드 장애 시에도 오토 티어링 서비스가 자동으로 재실행되는 고가용성을 확보